

## Imports y Funciones

```
In [2]: import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import cimcb_lite as cb

from scipy.stats import mannwhitneyu, gaussian_kde
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_predict
from sklearn.metrics import roc_curve, auc, roc_auc_score, confusion_matrix

# Importación de todos los modelos solicitados
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

from bokeh.io import output_notebook
from bokeh.plotting import figure, show
from bokeh.layouts import gridplot
from bokeh.models import Span, ColumnDataSource

output_notebook()
print("Entorno y paquetes cargados correctamente.")
```

 Loading BokehJS ...

Entorno y paquetes cargados correctamente.

## Métricas, Gráficos y Evaluación (ColumnDataSource)

```
In [3]: def safe_mannwhitneyu(pos, neg):
pos, neg = np.asarray(pos, dtype=float), np.asarray(neg, dtype=float)
pos, neg = pos[np.isfinite(pos)], neg[np.isfinite(neg)]
if len(pos) == 0 or len(neg) == 0: return np.nan
try: stat, p = mannwhitneyu(pos, neg, alternative='two-sided')
except TypeError: stat, p = mannwhitneyu(pos, neg); p = min(2 * p, 1.0)
return p

def metrics_from_scores(y_true, y_score, cutoff=0.5):
y_true, y_score = np.asarray(y_true), np.asarray(y_score, dtype=float)
mask = np.isfinite(y_score)
y_true, y_score = y_true[mask], y_score[mask]
```

```

if len(y_true) == 0: return {k: np.nan for k in ['Cut-off', 'Accuracy', 'Precision', 'Sensitivity', 'Specificity', 'F1-score', 'AUC', 'MW-U
y_pred = (y_score >= cutoff).astype(int)
tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
total = tn + fp + fn + tp
acc = (tp + tn) / total if total > 0 else np.nan
prec = tp / (tp + fp) if (tp + fp) > 0 else 0.0
sens = tp / (tp + fn) if (tp + fn) > 0 else 0.0
spec = tn / (tn + fp) if (tn + fp) > 0 else 0.0
f1 = 2*prec*sens/(prec+sens) if (prec+sens) > 0 else 0.0
try: auc_val = roc_auc_score(y_true, y_score)
except ValueError: auc_val = np.nan
p_mw = safe_mannwhitneyu(y_score[y_true == 1], y_score[y_true == 0])
return {'Cut-off': cutoff, 'Accuracy': acc, 'Precision': prec, 'Sensitivity': sens, 'Specificity': spec, 'F1-score': f1, 'AUC': auc_v

def metrics_table(model_name, y_tr, s_tr, y_te, s_te, cutoff=0.5):
tr_m, te_m = metrics_from_scores(y_tr, s_tr, cutoff), metrics_from_scores(y_te, s_te, cutoff)
tr_m.update({'Model': model_name, 'Set': 'Train'}); te_m.update({'Model': model_name, 'Set': 'Test'})
cols = ['Model', 'Set', 'Cut-off', 'Accuracy', 'Precision', 'Sensitivity', 'Specificity', 'F1-score', 'AUC', 'MW-U p-value']
return pd.DataFrame([tr_m, te_m])[cols]

def _boxplot_stats(vals):
vals = np.asarray(vals, dtype=float); vals = vals[np.isfinite(vals)]
if len(vals) == 0: return None
q1, q2, q3 = np.percentile(vals, [25, 50, 75]); iqr = q3 - q1
lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
inliers = vals[(vals >= lo) & (vals <= hi)]
return dict(q1=q1, q2=q2, q3=q3, lower=np.min(inliers) if len(inliers)>0 else q1, upper=np.max(inliers) if len(inliers)>0 else q3, ou

def plot_score_boxplot(y_true, y_score, cutoff=0.5, title='Score distribution', width=320, height=315):
y_true, y_score = np.asarray(y_true), np.asarray(y_score, dtype=float)
mask = np.isfinite(y_score); y_true, y_score = y_true[mask], y_score[mask]
labels = ['0 (HE)', '1 (GC)']; colors = {0: 'red', 1: 'blue'}
p = figure(x_range=labels, title=title, plot_width=width, plot_height=height, tools='pan,wheel_zoom,reset,save')
p.yaxis.axis_label = 'Predicted score'
groups, lowers, q1s, q2s, q3s, uppers = [], [], [], [], [], []
for cls, label in zip([0, 1], labels):
st = _boxplot_stats(y_score[y_true == cls])
if st: groups.append(label); lowers.append(st['lower']); q1s.append(st['q1']); q2s.append(st['q2']); q3s.append(st['q3']); uppers
box_src = ColumnDataSource(data=dict(group=groups, lower=lowers, q1=q1s, q2=q2s, q3=q3s, upper=uppers))
p.segment(x0='group', x1='group', y0='lower', y1='q1', source=box_src, line_color='black')
p.segment(x0='group', x1='group', y0='q3', y1='upper', source=box_src, line_color='black')
p.vbar(x='group', top='q3', bottom='q1', width=0.4, source=box_src, fill_color='gray', fill_alpha=0.35, line_color='black')
p.segment(x0='group', x1='group', y0='q2', y1='q2', source=box_src, line_color='black', line_width=2)
for cls, label in zip([0, 1], labels):
st = _boxplot_stats(y_score[y_true == cls])
if st and len(st['outliers']) > 0:
out_src = ColumnDataSource(data=dict(x=[label]*len(st['outliers']), y=st['outliers']))
p.circle(x='x', y='y', size=6, fill_color=colors[cls], line_color='black', source=out_src)
p.add_layout(Span(location=cutoff, dimension='width', line_color='black', line_dash='dashed', line_width=2))
return p

def plot_pdf(y_true, y_score, cutoff=0.5, title='PDF', width=320, height=315):

```

```

y_true, y_score = np.asarray(y_true), np.asarray(y_score, dtype=float)
mask = np.isfinite(y_score); y_true, y_score = y_true[mask], y_score[mask]
p = figure(title=title, plot_width=width, plot_height=height, tools='pan,wheel_zoom,reset,save')
p.xaxis.axis_label = 'Predicted score'; p.yaxis.axis_label = 'Density'
if len(y_score) == 0: return p
x_grid = np.linspace(y_score.min() - 0.05, y_score.max() + 0.05, 200)
for cls, color in zip([0, 1], ['red', 'blue']):
    vals = y_score[y_true == cls]
    if len(vals) < 2: continue
    try: dens = gaussian_kde(vals)(x_grid)
    except np.linalg.LinAlgError: continue
    src = ColumnDataSource(data=dict(x=x_grid, y=dens))
    p.patch('x', 'y', source=src, fill_color=color, fill_alpha=0.25, line_color=color)
    p.line('x', 'y', source=src, line_color=color, line_width=2)
p.add_layout(Span(location=cutoff, dimension='height', line_color='black', line_dash='dashed', line_width=2))
return p

def plot_roc(y_true, y_score, title='ROC', width=320, height=315):
    y_true, y_score = np.asarray(y_true), np.asarray(y_score, dtype=float)
    mask = np.isfinite(y_score); y_true, y_score = y_true[mask], y_score[mask]
    p = figure(title=title, plot_width=width, plot_height=height, tools='pan,wheel_zoom,reset,save')
    p.xaxis.axis_label = '1-Specificity'; p.yaxis.axis_label = 'Sensitivity'
    if len(np.unique(y_true)) < 2: return p
    fpr, tpr, _ = roc_curve(y_true, y_score); auc_val = auc(fpr, tpr)
    src = ColumnDataSource(data=dict(fpr=fpr, tpr=tpr))
    p.line('fpr', 'tpr', source=src, line_color='blue', line_width=2)
    p.line([0, 1], [0, 1], line_color='gray', line_dash='dashed')
    p.text(x=[0.45], y=[0.2], text=[f'AUC={auc_val:.3f}'], text_font_size='10pt')
    return p

def plot_signed_importance(values, feature_labels, title='Feature importance', width=1100, height=350, top_n=20):
    values, feature_labels = np.asarray(values, dtype=float), np.asarray(feature_labels)
    order = np.argsort(np.abs(values))[:-1][:top_n]
    feature_labels, values = feature_labels[order], values[order]
    src = ColumnDataSource(data=dict(x=feature_labels, top=np.maximum(values, 0), bottom=np.minimum(values, 0)))
    p = figure(x_range=list(feature_labels), title=title, plot_width=width, plot_height=height, tools='pan,wheel_zoom,reset,save')
    p.xaxis.major_label_orientation = np.pi / 2; p.yaxis.axis_label = 'Importance'
    p.vbar(x='x', top='top', bottom='bottom', width=0.6, source=src, fill_color='steelblue', line_color='black')
    return p

def _get_scores(model, X):
    if hasattr(model, 'predict_proba'): return model.predict_proba(X)[:, 1]
    elif hasattr(model, 'decision_function'): return model.decision_function(X)
    return model.predict(X) # Fallback para Regresión Lineal

def evaluate_sklearn_model(model, model_name, X_train, y_train, X_test, y_test, cutoff=0.5, cv=None):
    if cv:
        try: y_cv = cross_val_predict(model, X_train, y_train, cv=cv, method='predict_proba')[:, 1]
        except: y_cv = cross_val_predict(model, X_train, y_train, cv=cv, method='decision_function')
        show(plot_roc(y_train, y_cv, title=f'{model_name} 5-fold CV ROC'))
    model.fit(X_train, y_train)
    y_tr_s, y_te_s = _get_scores(model, X_train), _get_scores(model, X_test)

```

```

show(gridplot([plot_score_boxplot(y_train, y_tr_s, cutoff, f'{model_name} Train'), plot_pdf(y_train, y_tr_s, cutoff, f'{model_name}
                    [plot_score_boxplot(y_test, y_te_s, cutoff, f'{model_name} Test'), plot_pdf(y_test, y_te_s, cutoff, f'{model_name} Tes
df = metrics_table(model_name, y_train, y_tr_s, y_test, y_te_s, cutoff)
display(df)
return model, df, y_tr_s, y_te_s

def permutation_importance_custom(model, X, y, n_repeats=5, random_state=42):
    X, y = np.asarray(X, dtype=float), np.asarray(y)
    if len(np.unique(y)) < 2: return np.zeros(X.shape[1])
    base_score = roc_auc_score(y, _get_scores(model, X))
    rng, importances = np.random.RandomState(random_state), np.zeros(X.shape[1])
    for j in range(X.shape[1]):
        deltas = []
        for _ in range(n_repeats):
            X_perm = X.copy(); rng.shuffle(X_perm[:, j])
            try: sc = roc_auc_score(y, _get_scores(model, X_perm))
            except: sc = np.nan
            deltas.append(base_score - sc)
        importances[j] = np.nanmean(deltas)
    return importances

def get_and_plot_importance(model, model_name, X, y, peak_names, peak_labels, peak_plot_labels):
    if hasattr(model, 'feature_importances_'): imp, imp_type = model.feature_importances_, 'Native (Gini)'
    elif hasattr(model, 'coef_'): imp, imp_type = (model.coef_[0] if model.coef_.ndim > 1 else model.coef_), 'Coefficients'
    else: imp, imp_type = permutation_importance_custom(model, X, y), 'Permutation'
    df = pd.DataFrame({'Name': peak_names, 'Label': peak_labels, 'PlotLabel': peak_plot_labels, 'Importance': imp})
    df['Abs_Imp'] = np.abs(df['Importance'])
    df = df.sort_values('Abs_Imp', ascending=False).reset_index(drop=True)
    show(plot_signed_importance(df['Importance'].values, df['PlotLabel'].values, title=f'{model_name} ({imp_type})'))
    display(df.head(10))
    return df

```

## Carga y Limpieza de Datos

```

In [4]: filename = 'GastricCancer_NMR.xlsx'
dataTable, peakTable = cb.utils.load_dataXL(filename, DataSheet='Data', PeakSheet='Peak')

rsd, percMiss = peakTable['QC_RSD'], peakTable['Perc_missing']
peakTableClean = peakTable[(rsd < 20) & (percMiss < 10)].copy()
print("Number of peaks remaining:", len(peakTableClean))

peaklist = peakTableClean['Name'].tolist()
peak_names = np.array(peaklist)
peak_labels = peakTableClean.set_index('Name').loc[peaklist, 'Label'].astype(str).values
label_counts = pd.Series(peak_labels).value_counts()
peak_plot_labels = [lab if label_counts[lab] == 1 else f'{lab} ({name})' for name, lab in zip(peak_names, peak_labels)]

```

Loadings PeakFile: Peak  
Loadings DataFile: Data  
Data Table & Peak Table is suitable.  
TOTAL SAMPLES: 140 TOTAL PEAKS: 149  
Done!  
Number of peaks remaining: 52

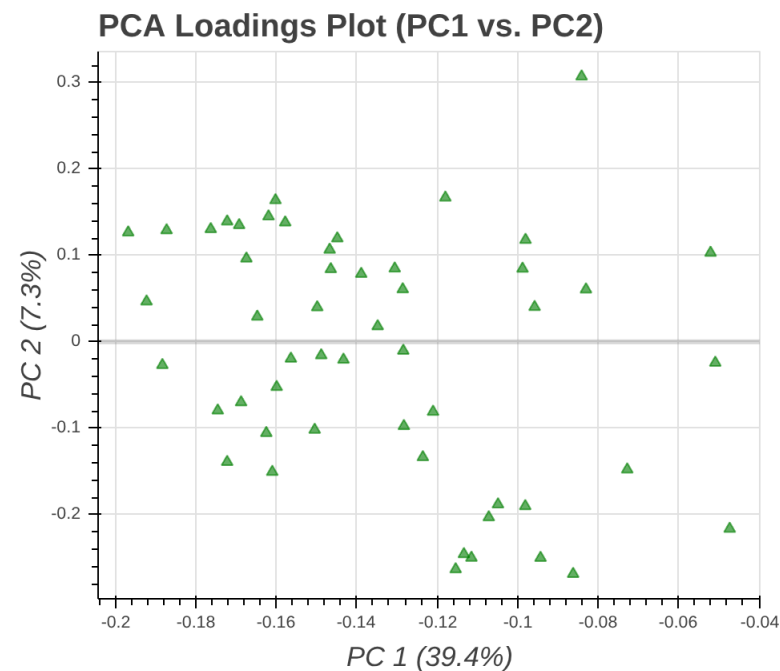
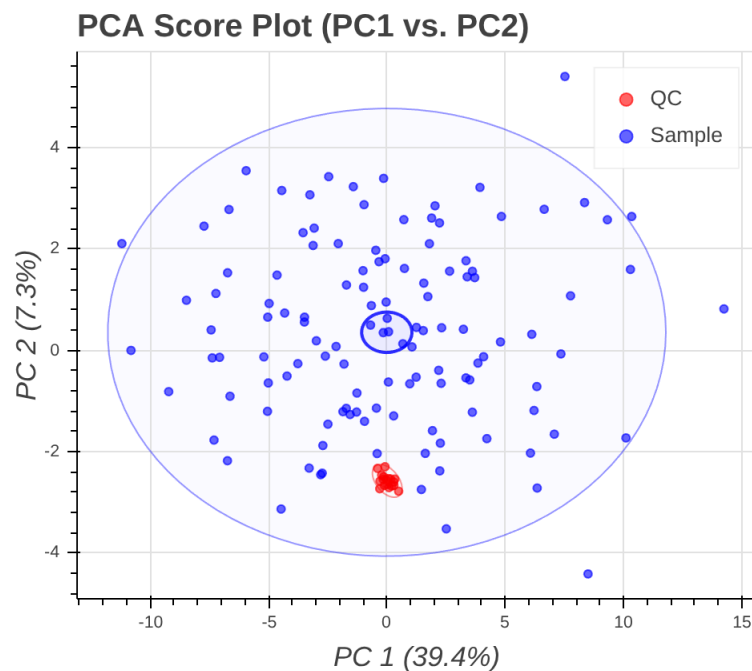
## PCA y Partición Train/Test

```
In [5]: X, Xlog = dataTable[peaklist].values, np.log10(dataTable[peaklist].values)
Xknn = cb.utils.knnimpute(cb.utils.scale(Xlog, method='auto'), k=3)
cb.plot.pca(Xknn, pcx=1, pcy=2, group_label=dataTable['SampleType'])

dataTable2 = dataTable[(dataTable.Class == "GC") | (dataTable.Class == "HE")].copy()
Y = np.array([1 if outcome == "GC" else 0 for outcome in dataTable2['Class']])
dataTrain, dataTest, Ytrain, Ytest = train_test_split(dataTable2, Y, test_size=0.25, stratify=Y, random_state=10)

XT, XV = dataTrain[peaklist].values, dataTest[peaklist].values
XTlog, XVlog = np.log(XT), np.log(XV)
mu, sigma = cb.utils.scale(XTlog, return_mu_sigma=True)
XTknn = cb.utils.knnimpute(cb.utils.scale(XTlog, method='auto'), k=3)
XVknn = cb.utils.knnimpute(cb.utils.scale(XVlog, method='auto', mu=mu, sigma=sigma), k=3)
```

 Loading BokehJS ...



### PCA Score Plot (PC1 vs. PC2) ¿Qué muestra?

Cada punto es una muestra. Rojo = QC (controles de calidad), azul = muestras experimentales. Los ejes PC1 y PC2 son componentes principales que resumen la variabilidad de los datos.

### ¿Cómo interpretarlo?

Los QC rojos están agrupados cerca del centro. Eso es buena señal: los controles son reproducibles.

Las muestras azules están más dispersas. Hay variabilidad biológica.

PC1 explica 39.4% de la varianza; PC2, 7.3%.

No se ve separación por clase GC/HE porque aquí solo se coloreó QC vs. Sample. PCA no sabe de clases; solo busca varianza.

El "PCA es como hacer un resumen visual del dataset. No está diseñado para clasificar, sino para ver estructura general."

**PCA Loadings Plot (PC1 vs. PC2) ¿Qué muestra?** Cada triángulo verde es un metabolito/pico. Su posición indica cuánto contribuye a PC1 y PC2.

### ¿Cómo interpretarlo?

Puntos lejos del origen (0,0) son los que más influyen en esos componentes.

Los que están cerca de 0 casi no aportan.

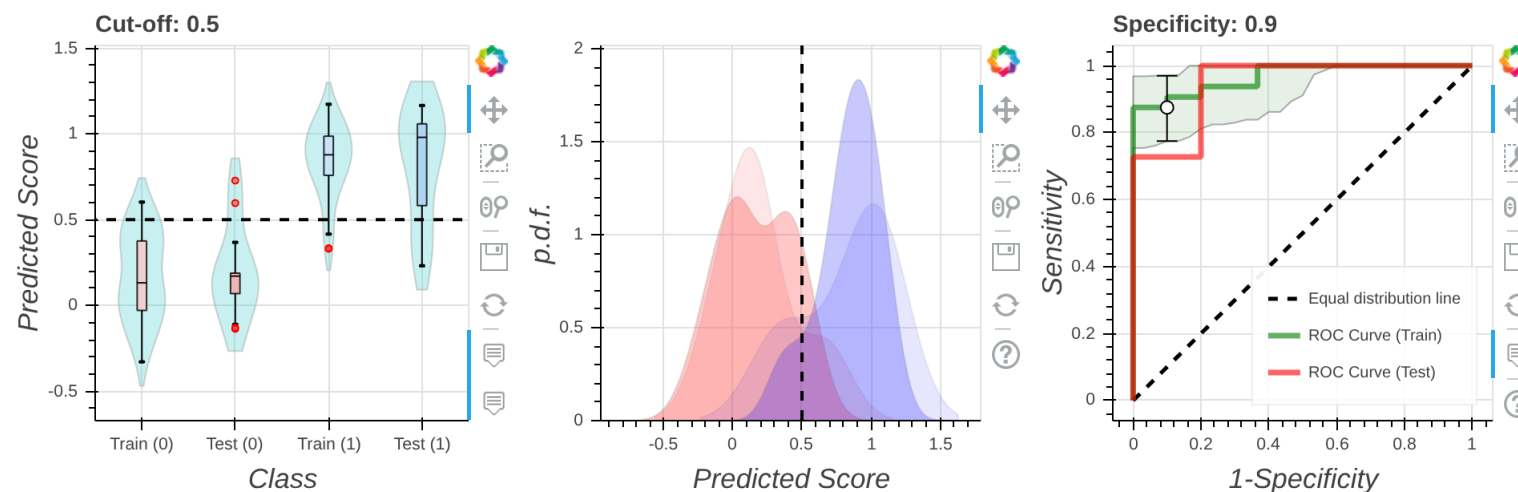
Sirve para explorar qué metabolitos mueven la variabilidad, pero no indica directamente qué separa GC de HE.

## Modelo Base: PLS-DA

```
In [6]: modelPLS = cb.model.PLS_SIMPLS(n_components=2)
Ypred_pls = modelPLS.train(XTknn, Ytrain)
YVpred_pls = modelPLS.test(XVknn)
modelPLS.evaluate([Ytest, YVpred_pls], cutoffscore=0.5)
metrics_pls = metrics_table('PLS-DA', Ytrain, Ypred_pls, Ytest, YVpred_pls, cutoff=0.5)
display(metrics_pls)
```

Loading BokehJS ...

Score cut-off fixed to: 0.5



| # | Evaluate | MW-U Pvalue | R2                | AUC               | Accuracy          | Precision        | Sensitivity       | F1score           |
|---|----------|-------------|-------------------|-------------------|-------------------|------------------|-------------------|-------------------|
| 0 | Train    | 2.66e-10    | 0.67 (0.55, 0.76) | 0.97 (0.92, 0.99) | 0.89 (0.84, 0.94) | 0.9 (0.84, 0.95) | 0.88 (0.77, 0.97) | 0.89 (0.82, 0.95) |
| 1 | Test     | 6.37e-04    | 0.52              | 0.95              | 0.76              | 0.8              | 0.73              | 0.76              |

|   | Model  | Set   | Cut-off | Accuracy | Precision | Sensitivity | Specificity | F1-score | AUC      | MW-U p-value |
|---|--------|-------|---------|----------|-----------|-------------|-------------|----------|----------|--------------|
| 0 | PLS-DA | Train | 0.5     | 0.887097 | 0.903226  | 0.875000    | 0.9         | 0.888889 | 0.967708 | 2.657802e-10 |
| 1 | PLS-DA | Test  | 0.5     | 0.761905 | 0.800000  | 0.727273    | 0.8         | 0.761905 | 0.945455 | 6.371949e-04 |

**Modelo PLS-DA****Boxplot (izquierda):**

Eje X: Train(0), Test(0), Train(1), Test(1).

Eje Y: score predicho.

Línea punteada: cut-off 0.5.

**Train:** clase 0 debajo de 0.5, clase 1 arriba. Buena separación.

**Test:** clase 0 y 1 se acercan más al cut-off. Hay más solapamiento.

**(centro):**

Curva roja = HE; curva azul = GC.

Train: picos bien separados.

Test: las curvas se solapan un poco más.

**ROC (derecha):**

Curva verde = Train; curva roja = Test.

AUC Train  $\approx$  0.968; AUC Test  $\approx$  0.945.

Muy buen rendimiento, aunque baja un poco en test.

**Tabla:**

**Train:** Accuracy 0.887, Sensibilidad 0.875, Especificidad 0.90, F1 0.889, AUC 0.968.

**Test:** Accuracy 0.762, Sensibilidad 0.727, Especificidad 0.80, F1 0.762, AUC 0.945.

**Interpretación:** PLS-DA funciona bien, pero hay cierta pérdida al generalizar. Es un modelo clásico y robusto en metabolómica.

## Modelos Complementarios de Machine Learning

### Regresión Lineal (Linear Probability Model)

No recomendado para clasificación binaria real, pero útil como baseline (modelo de referencia) matemático.

```

In [19]: def _get_predict_method(model):
    """Detecta el método de predicción adecuado para cada modelo."""
    if hasattr(model, 'predict_proba'):
        return 'predict_proba'
    elif hasattr(model, 'decision_function'):
        return 'decision_function'
    else:
        return 'predict' # Para LinearRegression y otros regresores

def _get_scores(model, X):
    """Obtiene scores de clase positiva según el tipo de modelo."""
    if hasattr(model, 'predict_proba'):
        return model.predict_proba(X)[: , 1]
    elif hasattr(model, 'decision_function'):
        return model.decision_function(X)
    else:
        return model.predict(X) # LinearRegression devuelve valores continuos

def evaluate_sklearn_model(model, model_name, X_train, y_train,
                           X_test, y_test, cutoff=0.5, cv=None):
    """Evalúa cualquier modelo sklearn (clasificadores Y regresores)."""
    if cv is not None:
        method = _get_predict_method(model)
        try:
            if method == 'predict_proba':
                y_cv = cross_val_predict(model, X_train, y_train, cv=cv,
                                         method='predict_proba')[: , 1]
            else:
                y_cv = cross_val_predict(model, X_train, y_train, cv=cv,
                                         method=method)
        except Exception as e:
            print(f"Advertencia CV para {model_name}: {e}")
            y_cv = None

        if y_cv is not None:
            show(plot_roc(y_train, y_cv, title=f'{model_name} 5-fold CV ROC'))

    model.fit(X_train, y_train)

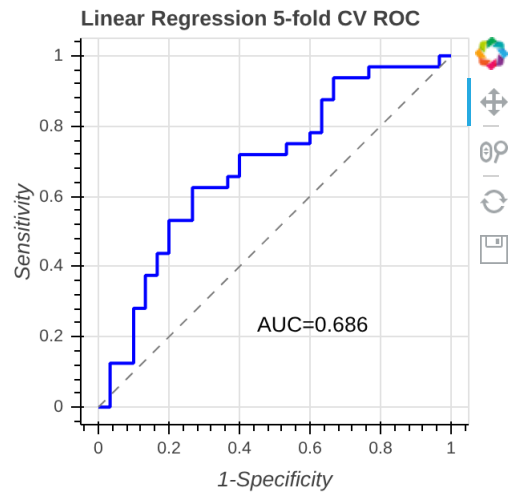
    y_train_score = _get_scores(model, X_train)
    y_test_score = _get_scores(model, X_test)

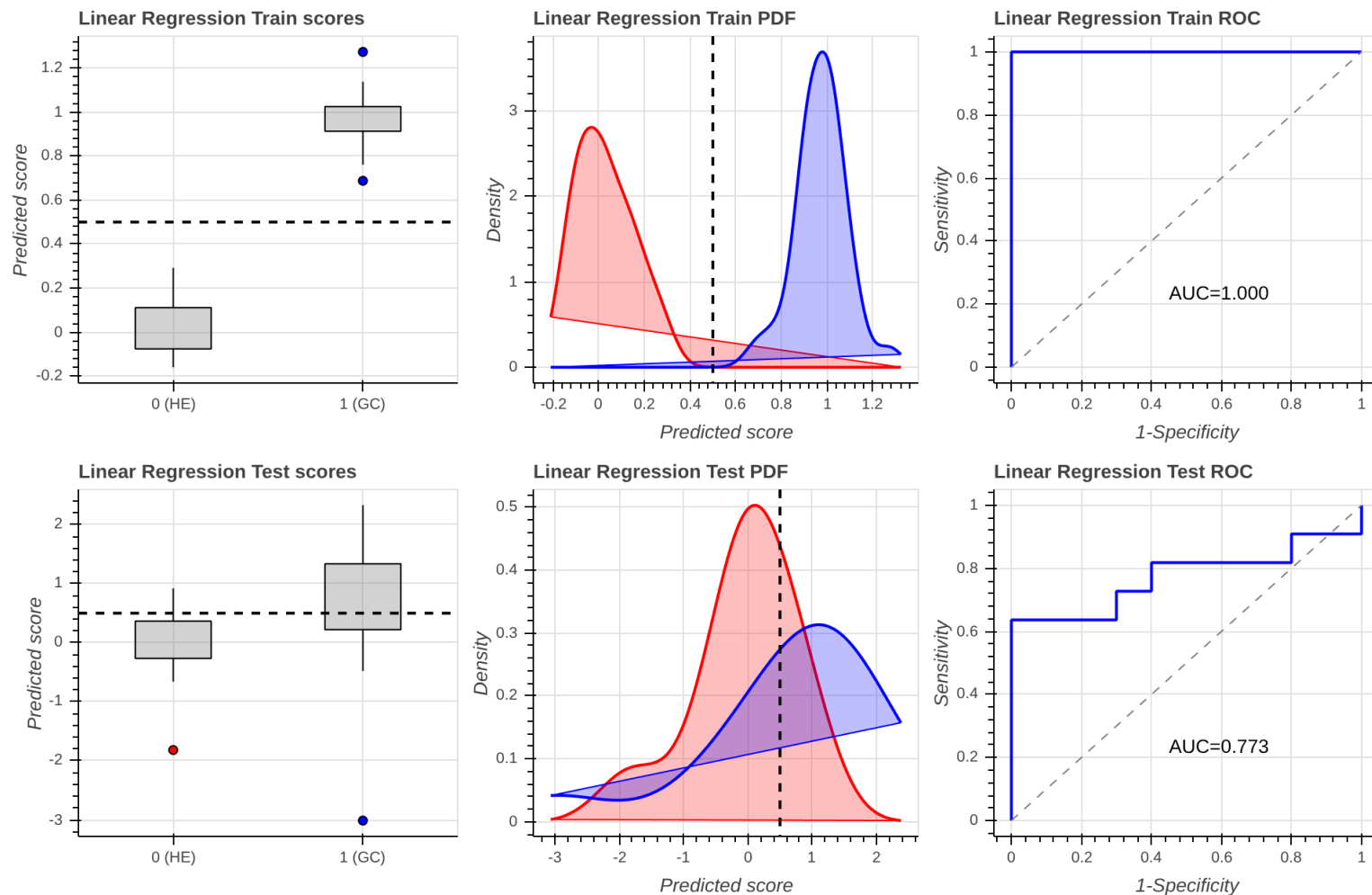
    show(gridplot([
        plot_score_boxplot(y_train, y_train_score, cutoff, title=f'{model_name} Train scores'),
        plot_pdf(y_train, y_train_score, cutoff, title=f'{model_name} Train PDF'),
        plot_roc(y_train, y_train_score, title=f'{model_name} Train ROC')
    ], [
        plot_score_boxplot(y_test, y_test_score, cutoff, title=f'{model_name} Test scores'),
        plot_pdf(y_test, y_test_score, cutoff, title=f'{model_name} Test PDF'),
        plot_roc(y_test, y_test_score, title=f'{model_name} Test ROC')
    ]))

```

```
l))\n\nmetrics_df = metrics_table(model_name, y_train, y_train_score,\n                           y_test, y_test_score, cutoff)\n\ndisplay(metrics_df)\n\nreturn model, metrics_df, y_train_score, y_test_score
```

```
In [20]: # Regresión Lineal\n        cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)\n        model_lin = LinearRegression()\n        model_lin, metrics_lin, y_tr_lin, y_te_lin = evaluate_sklern_model(\n            model_lin, 'Linear Regression', XTknn, Ytrain, XVknn, Ytest, cv=cv)
```





|   | Model             | Set   | Cut-off | Accuracy | Precision | Sensitivity | Specificity | F1-score | AUC      | MW-U p-value |
|---|-------------------|-------|---------|----------|-----------|-------------|-------------|----------|----------|--------------|
| 0 | Linear Regression | Train | 0.5     | 1.000000 | 1.000000  | 1.000000    | 1.0         | 1.0      | 1.000000 | 1.436343e-11 |
| 1 | Linear Regression | Test  | 0.5     | 0.714286 | 0.777778  | 0.636364    | 0.8         | 0.7      | 0.772727 | 3.777135e-02 |

### Regresión Lineal 5-fold CV ROC

¿Qué muestra? Curva ROC obtenida con validación cruzada de 5 pliegues. AUC = 0.686.

¿Cómo interpretarlo?

La curva está relativamente cerca de la diagonal. AUC 0.686 = capacidad discriminativa pobre-moderada. La regresión lineal no es un clasificador ideal; se usa solo como baseline.

### Regresión Lineal Train/Test

#### Train (fila superior):

Boxplot: clase 0 alrededor de 0, clase 1 alrededor de 1. Muy separadas.

PDF: curvas roja y azul casi sin solape.

ROC: AUC = 1.000. Perfecto en train.

#### Test (fila inferior):

Boxplot: hay solapamiento y valores atípicos.

**Centro:** curvas se cruzan bastante.

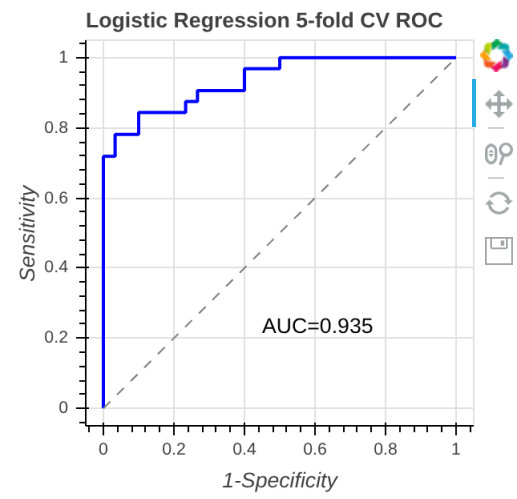
ROC: AUC = 0.773.

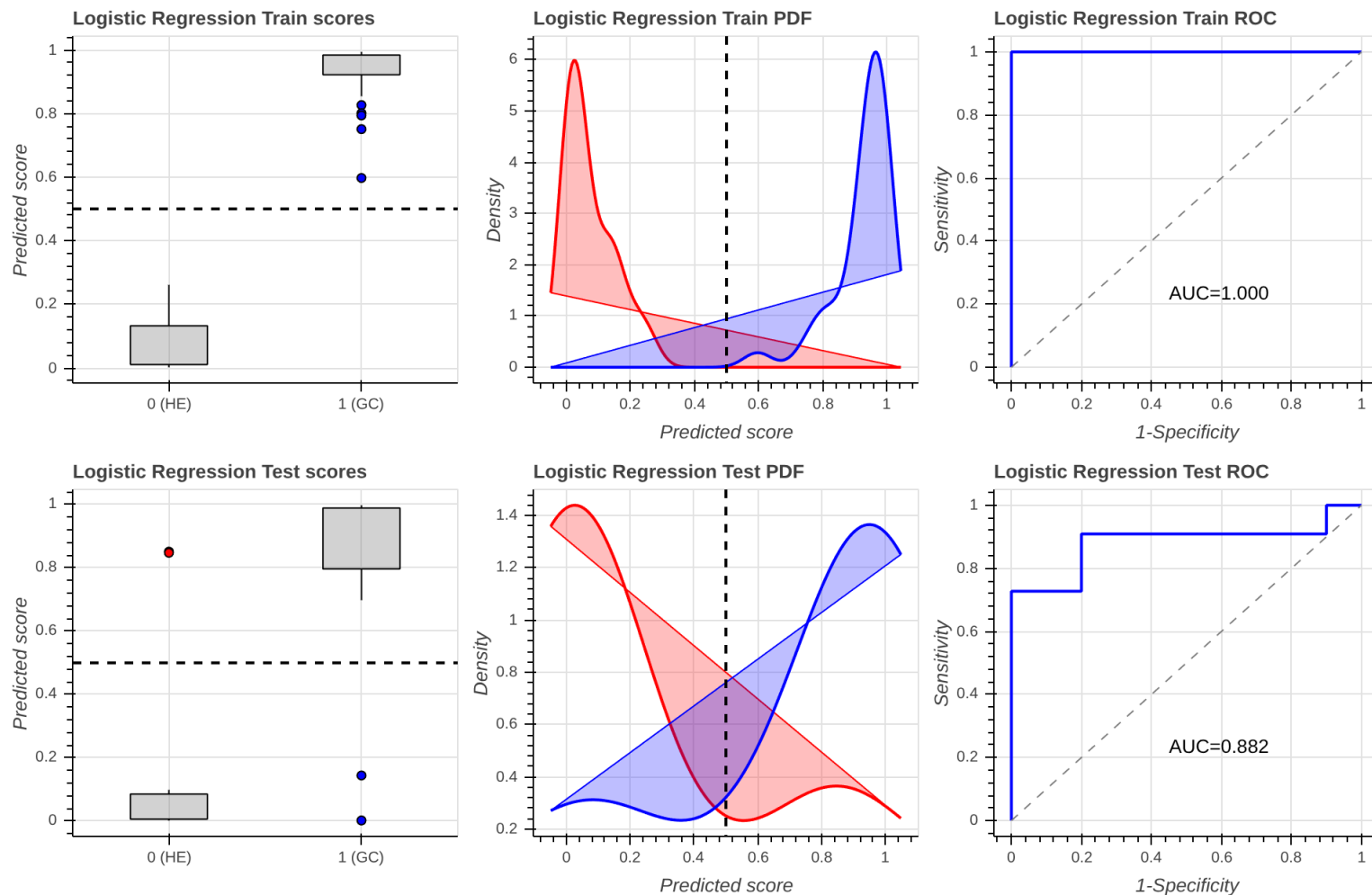
**Tabla:** Test Accuracy 0.714, Sensibilidad 0.636, Especificidad 0.80, F1 0.700, AUC 0.773.

**Interpretación:** Claro sobreajuste. En train es perfecto, en test baja mucho. No recomendado para clasificación binaria real.

## Regresión Logística

```
In [21]: # Regresión Logística
model_log = LogisticRegression(solver='lbfgs', class_weight='balanced', random_state=42)
model_log, metrics_log, y_tr_log, y_te_log = evaluate_sklern_model(
    model_log, 'Logistic Regression', XTknn, Ytrain, XVknn, Ytest, cv=cv)
```





|   | Model               | Set   | Cut-off | Accuracy | Precision | Sensitivity | Specificity | F1-score | AUC      | MW-U p-value |
|---|---------------------|-------|---------|----------|-----------|-------------|-------------|----------|----------|--------------|
| 0 | Logistic Regression | Train | 0.5     | 1.000000 | 1.000000  | 1.000000    | 1.0         | 1.000000 | 1.000000 | 1.436343e-11 |
| 1 | Logistic Regression | Test  | 0.5     | 0.809524 | 0.818182  | 0.818182    | 0.8         | 0.818182 | 0.881818 | 3.474099e-03 |

### Regresión Logística

Regresión Logística 5-fold CV ROC

AUC = 0.935. Curva muy cerca de la esquina superior izquierda. Excelente capacidad discriminativa en validación cruzada.

### Regresión Logística Train/Test

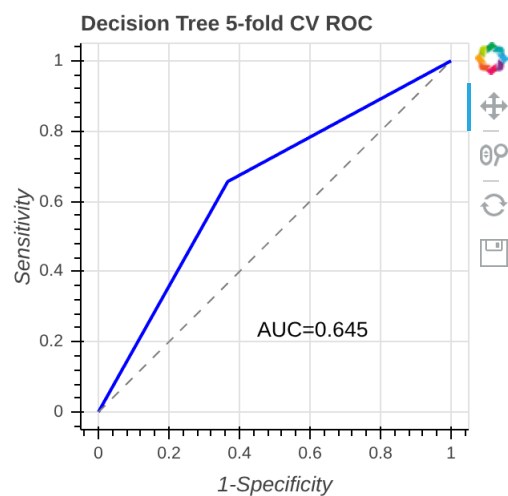
**Train:** boxplot, PDF y ROC perfectos (AUC = 1.000). **Test:** AUC = 0.882.

**Tabla:** Accuracy 0.810, Sensibilidad 0.818, Especificidad 0.80, F1 0.818, AUC 0.882.

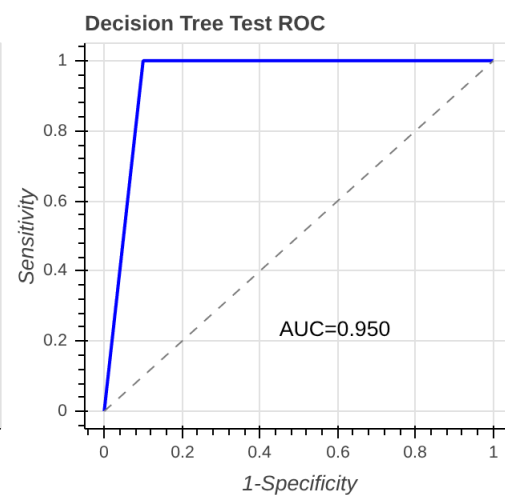
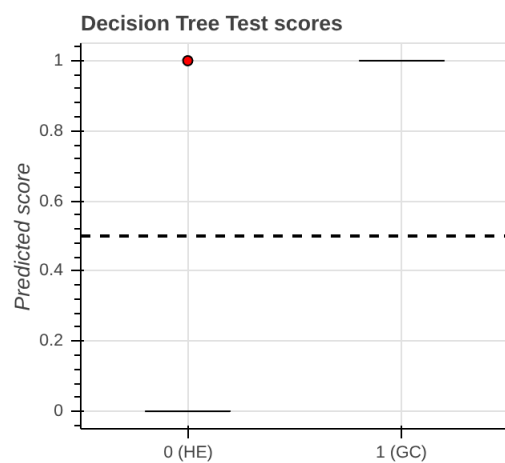
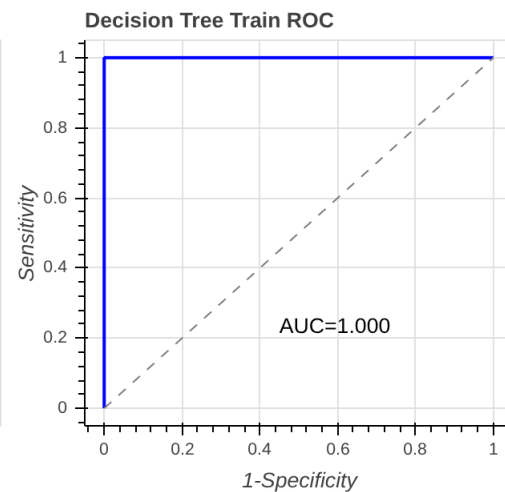
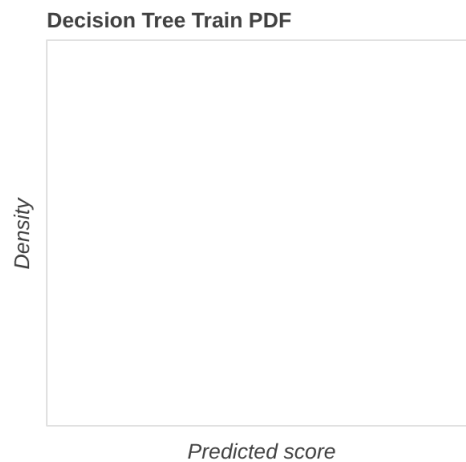
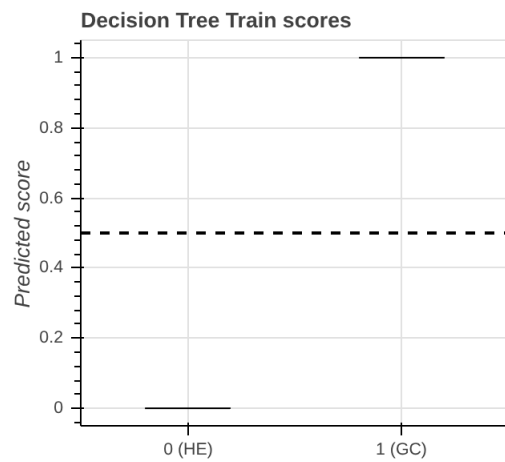
**Interpretación:** Muy buen modelo. Generaliza bien. Es uno de los mejores en relación simplicidad/rendimiento.

## Árboles de Decisión

```
In [22]: # Árbol de Decisión
model_tree = DecisionTreeClassifier(class_weight='balanced', random_state=42)
model_tree, metrics_tree, y_tr_tree, y_te_tree = evaluate_skl_model(
    model_tree, 'Decision Tree', XTknn, Ytrain, XVknn, Ytest, cv=cv)
```



WARNING:bokeh.core.validation.check:W-1000 (MISSING\_RENDERERS): Plot has no renderers: Figure(id='22238', ...)



|   | Model         | Set   | Cut-off | Accuracy | Precision | Sensitivity | Specificity | F1-score | AUC  | MW-U p-value |
|---|---------------|-------|---------|----------|-----------|-------------|-------------|----------|------|--------------|
| 0 | Decision Tree | Train | 0.5     | 1.000000 | 1.000000  | 1.0         | 1.0         | 1.000000 | 1.00 | 6.087861e-15 |
| 1 | Decision Tree | Test  | 0.5     | 0.952381 | 0.916667  | 1.0         | 0.9         | 0.956522 | 0.95 | 5.795438e-05 |

### Árbol de Decisión 5-fold CV ROC

AUC = 0.645. Curva pobre. El árbol no logra buena separación en validación cruzada. Además aparece un warning de Bokeh, pero la curva se

### Árbol de Decisión Train/Test

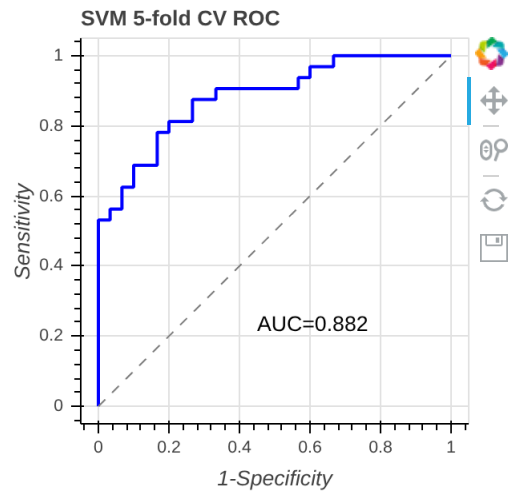
Train: scores discretos (0 o 1), PDF extraña porque los valores son binarios. ROC AUC = 1.000. Test: AUC = 0.950.

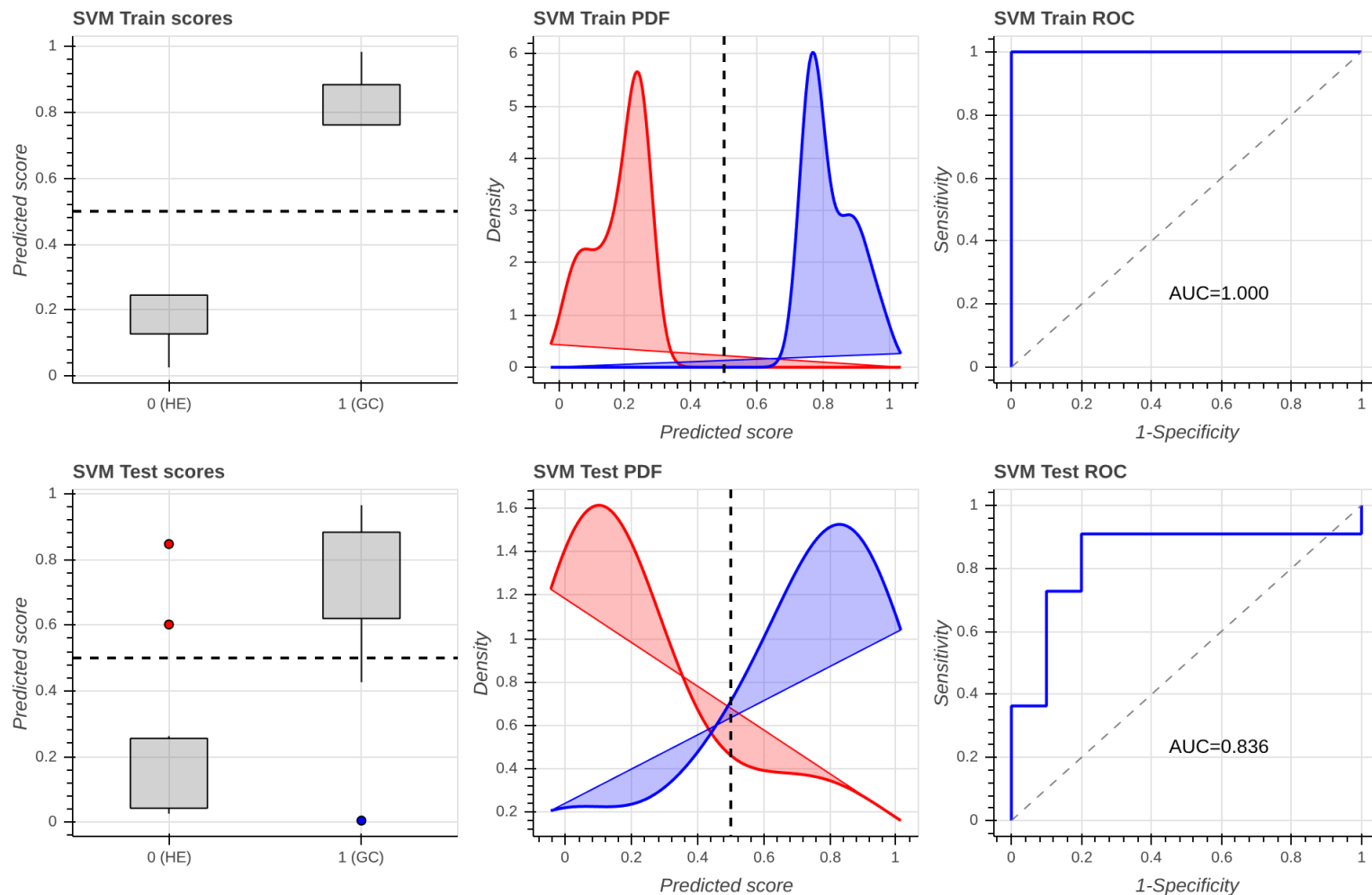
Tabla: Test Accuracy 0.952, Sensibilidad 1.000, Especificidad 0.90, F1 0.956, AUC 0.950.

**Interpretación:** El árbol aprende perfecto en train, pero el CV dice AUC 0.645. El buen resultado en test puede ser casualidad por el conjunto pequeño. Los árboles son inestables. Hay que tener cuidado.

## Support Vector Machine (SVM)

```
In [23]: # SVM
model_svm = SVC(kernel='linear', C=1.0, class_weight='balanced',
                 probability=True, random_state=42)
model_svm, metrics_svm, y_tr_svm, y_te_svm = evaluate_sklern_model(
    model_svm, 'SVM', XTknn, Ytrain, XVknn, Ytest, cv=cv)
```





| Model | Set | Cut-off | Accuracy | Precision | Sensitivity | Specificity | F1-score | AUC      | MW-U p-value |
|-------|-----|---------|----------|-----------|-------------|-------------|----------|----------|--------------|
| 0     | SVM | Train   | 0.5      | 1.000000  | 1.000000    | 1.000000    | 1.0      | 1.000000 | 1.436343e-11 |
| 1     | SVM | Test    | 0.5      | 0.809524  | 0.818182    | 0.818182    | 0.8      | 0.836364 | 1.016244e-02 |

### SVM (Support Vector Machine)

SVM 5-fold CV ROC AUC = 0.882. Buen rendimiento en validación cruzada.

### SVM Train/Test

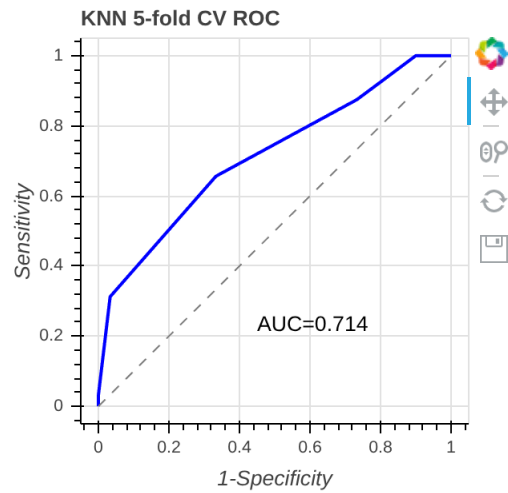
Train: AUC = 1.000. Test: AUC = 0.836.

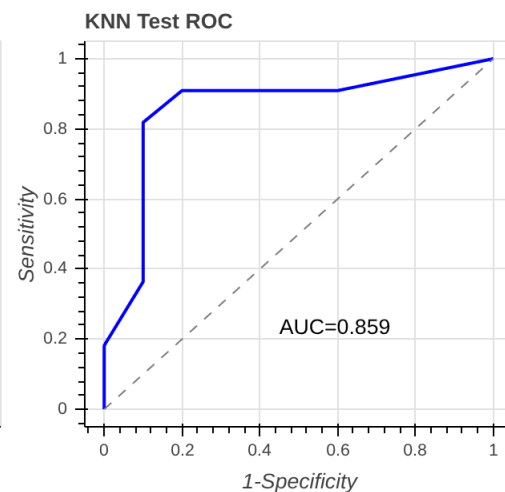
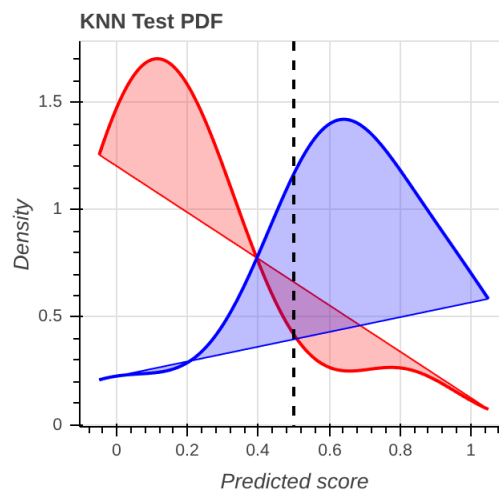
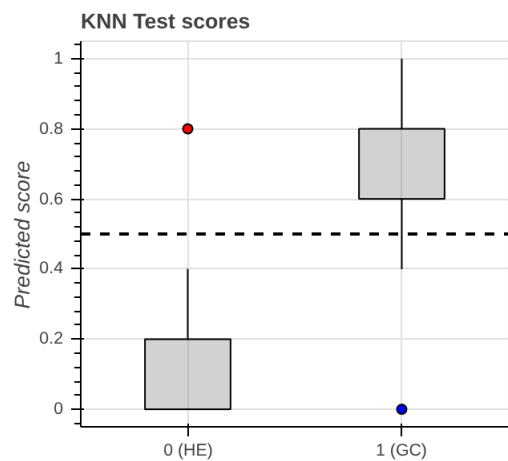
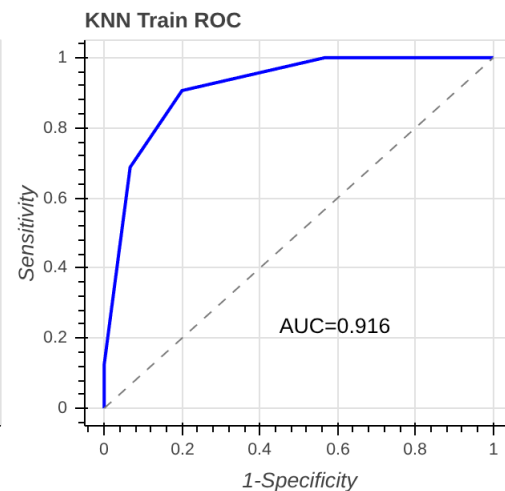
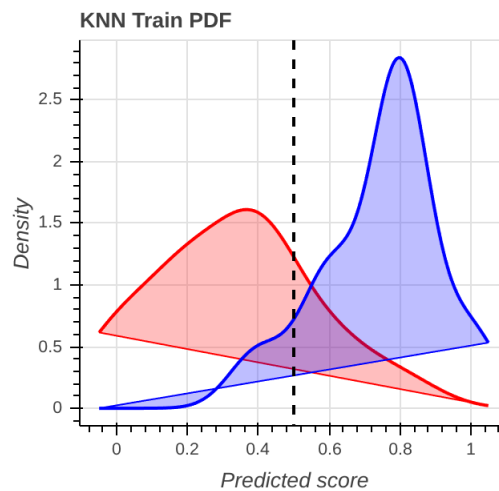
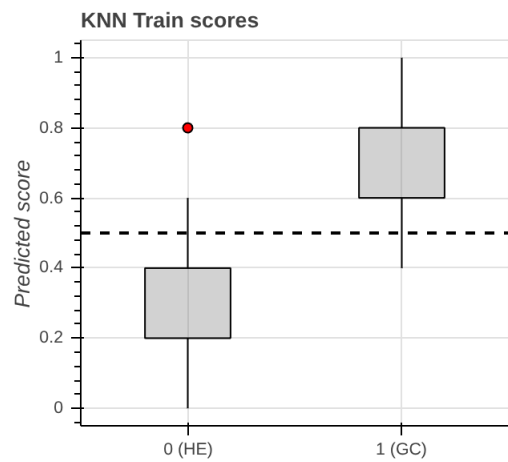
Tabla: Test Accuracy 0.810, Sensibilidad 0.818, Especificidad 0.80, F1 0.818, AUC 0.836.

**Interpretación:** Buen modelo, aunque con algo de sobreajuste. SVM lineal funciona bien.

## K-Nearest Neighbors (KNN)

```
In [24]: # KNN
model_knn = KNeighborsClassifier(n_neighbors=5)
model_knn, metrics_knn, y_tr_knn, y_te_knn = evaluate_sklern_model(
    model_knn, 'KNN', XTknn, Ytrain, XVknn, Ytest, cv=cv)
```





|   | Model | Set   | Cut-off | Accuracy | Precision | Sensitivity | Specificity | F1-score | AUC      | MW-U p-value |
|---|-------|-------|---------|----------|-----------|-------------|-------------|----------|----------|--------------|
| 0 | KNN   | Train | 0.5     | 0.854839 | 0.828571  | 0.906250    | 0.8         | 0.865672 | 0.916146 | 7.626721e-09 |
| 1 | KNN   | Test  | 0.5     | 0.857143 | 0.900000  | 0.818182    | 0.9         | 0.857143 | 0.859091 | 5.147835e-03 |

### KNN (K-Nearest Neighbors)

KNN 5-fold CV ROC AUC = 0.714. Rendimiento moderado.

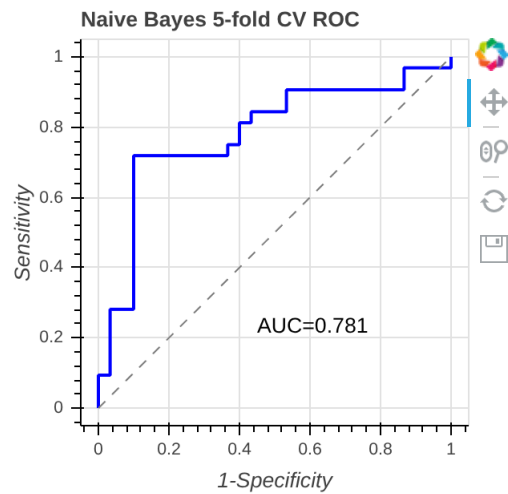
**KNN Train/Test** Train: AUC = 0.916. Test: AUC = 0.859.

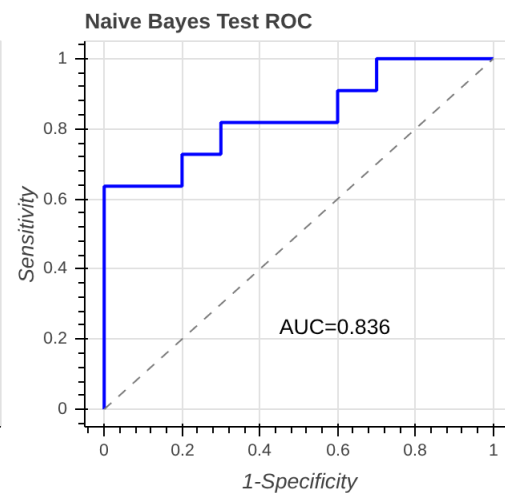
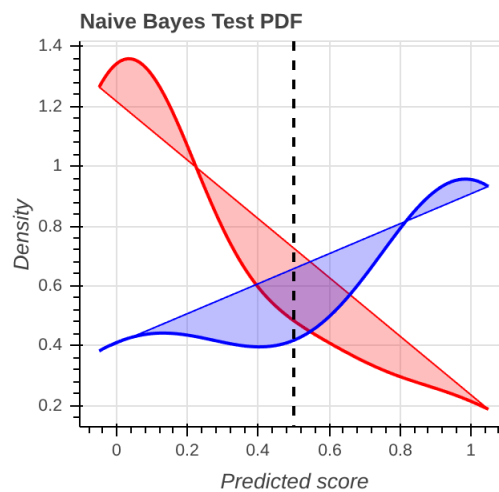
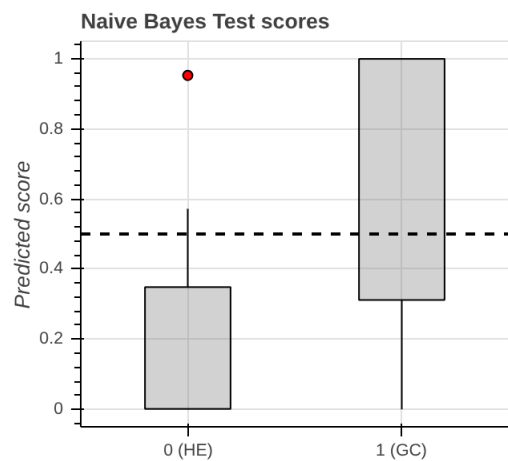
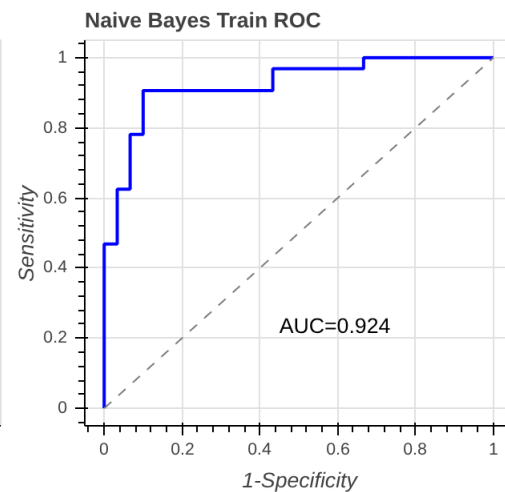
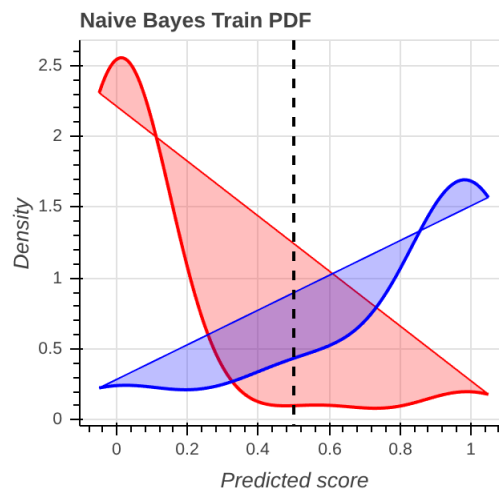
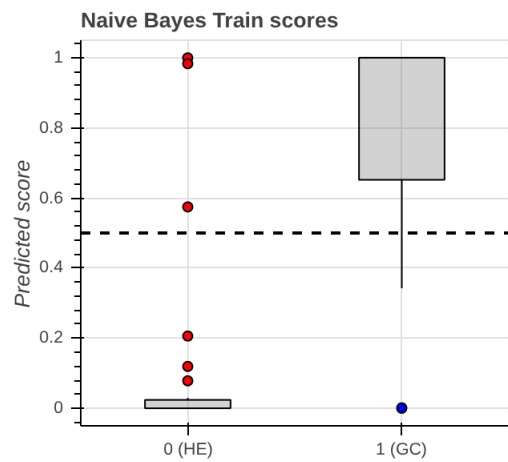
**Tabla:** Test Accuracy 0.857, Sensibilidad 0.818, Especificidad 0.90, F1 0.857, AUC 0.859.

**Interpretación:** KNN funciona aceptablemente. No es el mejor, pero cumple

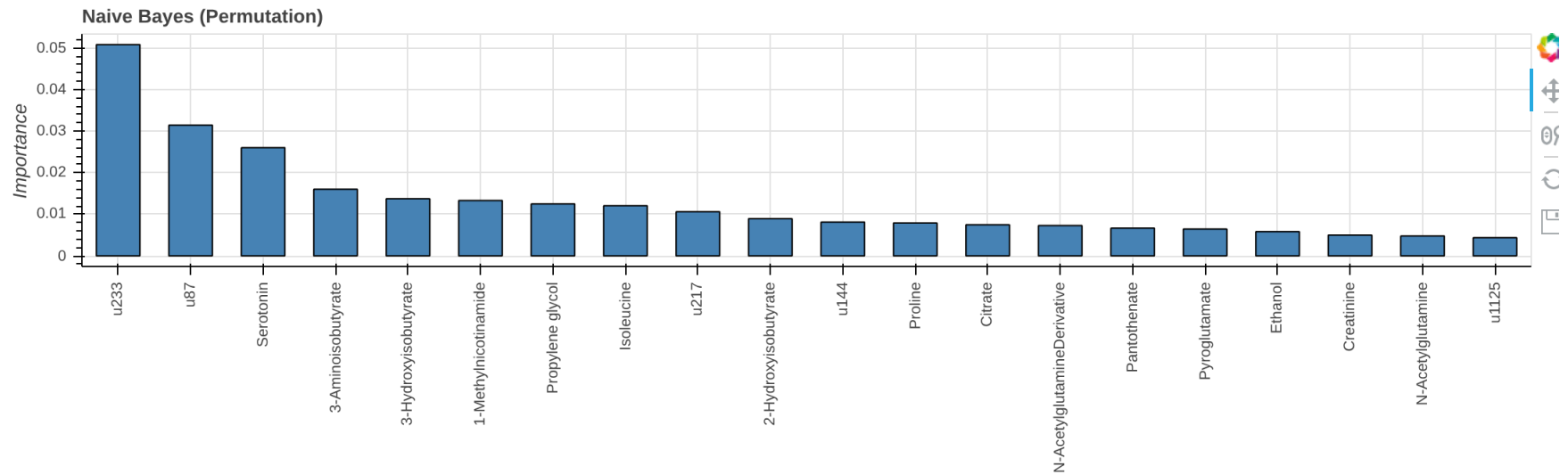
## Naive Bayes

```
In [25]: model_nb = GaussianNB()  
model_nb, metrics_nb, y_tr_nb, y_te_nb = evaluate_sklern_model(model_nb, 'Naive Bayes', XTknn, Ytrain, XVknn, Ytest, cv=cv)  
df_imp_nb = get_and_plot_importance(model_nb, 'Naive Bayes', XTknn, Ytrain, peak_names, peak_labels, peak_plot_labels)
```





|   | Model       | Set   | Cut-off | Accuracy | Precision | Sensitivity | Specificity | F1-score | AUC      | MW-U p-value |
|---|-------------|-------|---------|----------|-----------|-------------|-------------|----------|----------|--------------|
| 0 | Naive Bayes | Train | 0.5     | 0.854839 | 0.896552  | 0.812500    | 0.9         | 0.852459 | 0.923958 | 1.024385e-08 |
| 1 | Naive Bayes | Test  | 0.5     | 0.714286 | 0.777778  | 0.636364    | 0.8         | 0.700000 | 0.836364 | 1.016244e-02 |



|   | Name | Label                | PlotLabel            | Importance | Abs_Imp  |
|---|------|----------------------|----------------------|------------|----------|
| 0 | M138 | u233                 | u233                 | 0.050833   | 0.050833 |
| 1 | M144 | u87                  | u87                  | 0.031458   | 0.031458 |
| 2 | M110 | Serotonin            | Serotonin            | 0.026042   | 0.026042 |
| 3 | M11  | 3-Aminoisobutyrate   | 3-Aminoisobutyrate   | 0.016042   | 0.016042 |
| 4 | M14  | 3-Hydroxyisobutyrate | 3-Hydroxyisobutyrate | 0.013750   | 0.013750 |
| 5 | M4   | 1-Methylnicotinamide | 1-Methylnicotinamide | 0.013333   | 0.013333 |
| 6 | M105 | Propylene glycol     | Propylene glycol     | 0.012500   | 0.012500 |
| 7 | M74  | Isoleucine           | Isoleucine           | 0.012083   | 0.012083 |
| 8 | M137 | u217                 | u217                 | 0.010625   | 0.010625 |
| 9 | M10  | 2-Hydroxyisobutyrate | 2-Hydroxyisobutyrate | 0.008333   | 0.008333 |

### Naive Bayes

Naive Bayes 5-fold CV ROC AUC = 0.781. Rendimiento moderado.

**Naive Bayes Train/Test** Train: AUC = 0.924. Test: AUC = 0.836.

**Tabla:** Test Accuracy 0.714, Sensibilidad 0.636, Especificidad 0.80, F1 0.700, AUC 0.836.

**Interpretación:** En test baja bastante. Naive Bayes asume independencia entre variables, lo cual no siempre se cumple en metabolómica.

### Naive Bayes Permutation Importance

¿Qué muestra? Barras horizontales con los 20 metabolitos más importantes según permutation importance.

Eje Y: importancia.

Eje X: nombre del metabolito/pico.

¿Cómo interpretarlo?

Barra más alta = al desordenar esa variable, el AUC cae más; por tanto, es más importante para el modelo.

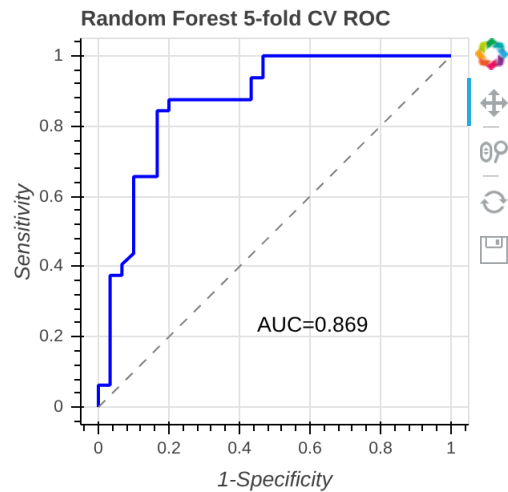
Top: u233, u87, Serotonin, 3-Aminoisobutyrate, 3-Hydroxyisobutyrate, etc.

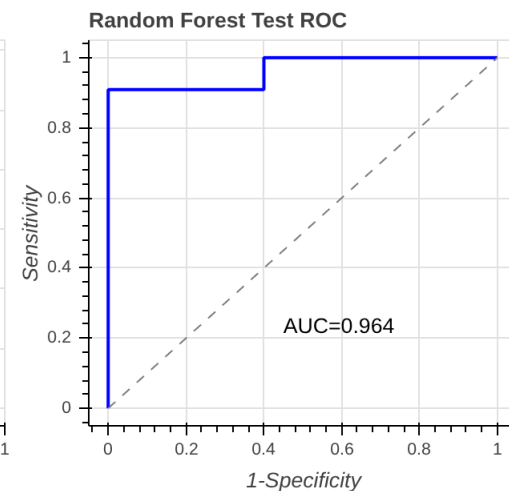
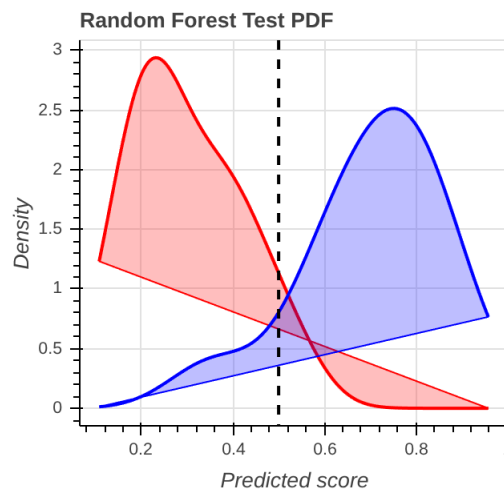
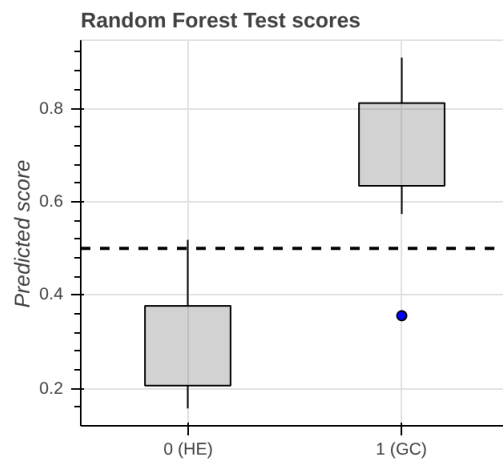
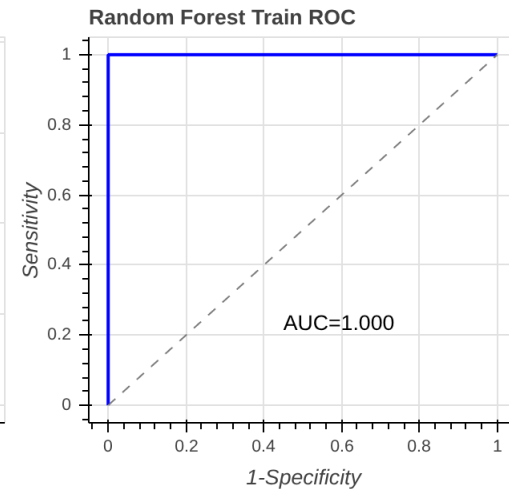
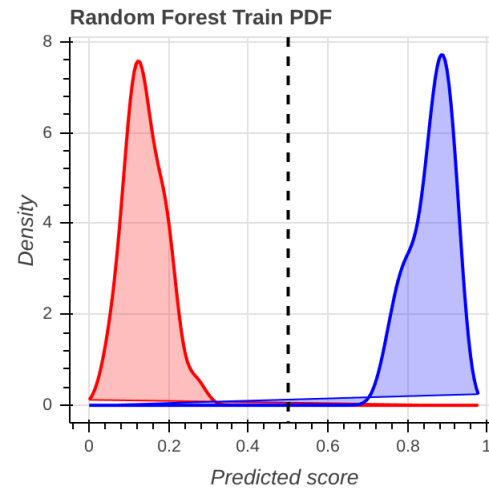
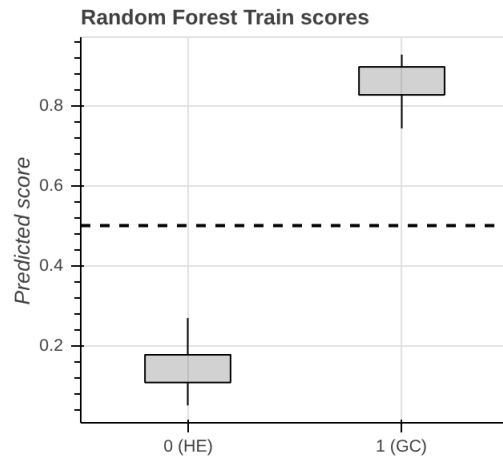
La tabla debajo lista nombre, etiqueta, importancia y valor absoluto.

“Esto no significa causalidad, sino que el modelo usa mucho esas variables para predecir.”

## Bosque Aleatorio (Random Forest)

```
In [26]: # Random Forest
model_rf = RandomForestClassifier(n_estimators=500, class_weight='balanced',
                                random_state=42, n_jobs=1)
model_rf, metrics_rf, y_tr_rf, y_te_rf = evaluate_sklern_model(
    model_rf, 'Random Forest', XTknn, Ytrain, XVknn, Ytest, cv=cv)
```





|   | Model         | Set   | Cut-off | Accuracy | Precision | Sensitivity | Specificity | F1-score | AUC      | MW-U p-value |
|---|---------------|-------|---------|----------|-----------|-------------|-------------|----------|----------|--------------|
| 0 | Random Forest | Train | 0.5     | 1.000000 | 1.000000  | 1.000000    | 1.0         | 1.000000 | 1.000000 | 1.432978e-11 |
| 1 | Random Forest | Test  | 0.5     | 0.904762 | 0.909091  | 0.909091    | 0.9         | 0.909091 | 0.963636 | 3.763967e-04 |

### Random Forest 5-fold CV ROC

AUC = 0.869. Buen rendimiento.

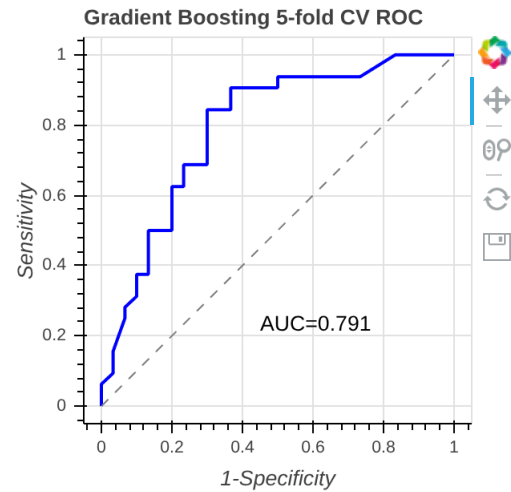
**Random Forest Train/Test** Train: AUC = 1.000. Test: AUC = 0.964.

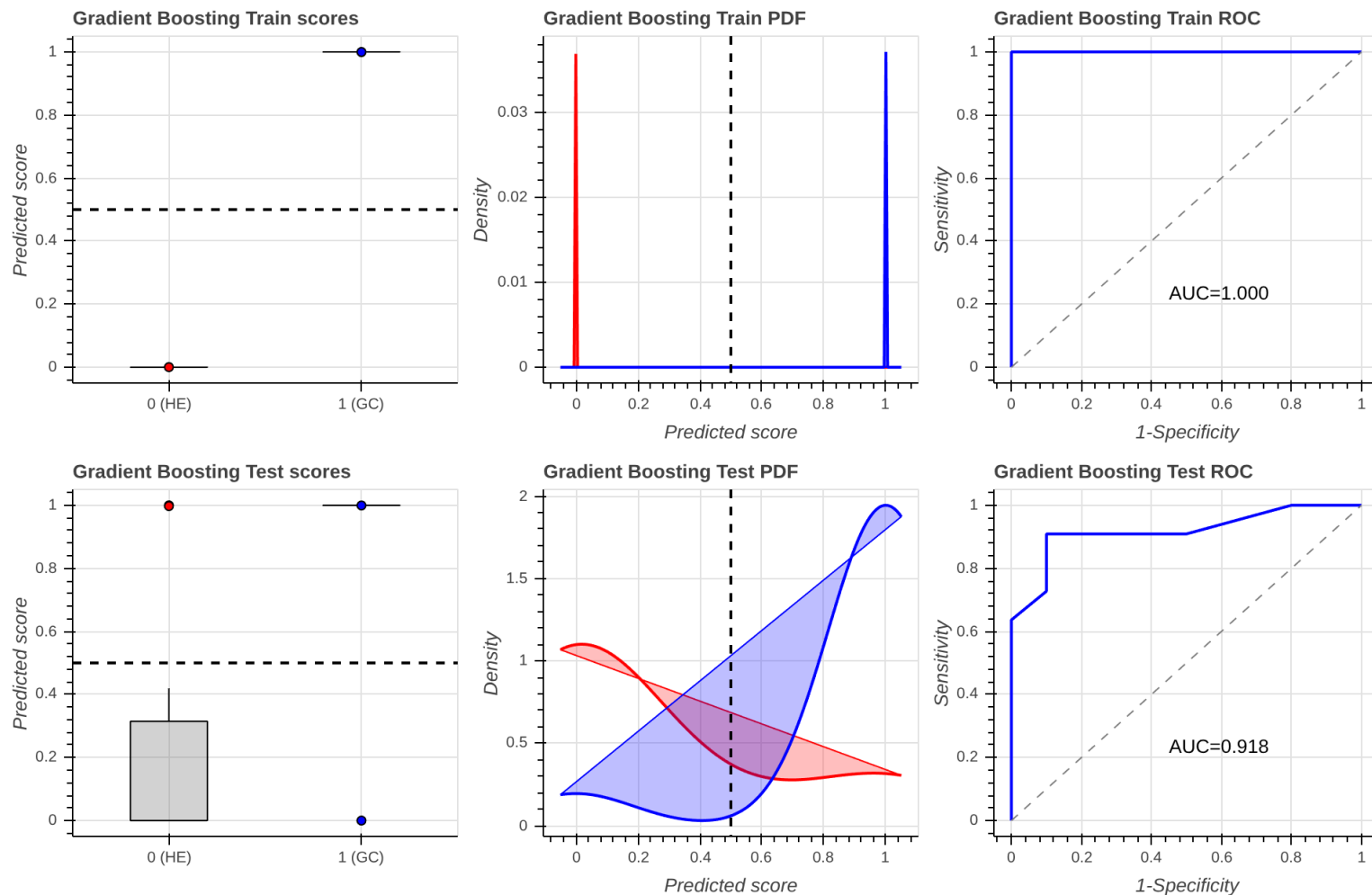
**Tabla:** Test Accuracy 0.905, Sensibilidad 0.909, Especificidad 0.90, F1 0.909, AUC 0.964.

Interpretación: Es el mejor modelo en test. Muy buena capacidad de generalización. Random Forest maneja bien muchas variables y no linealidades.

## Gradient Boosting

```
In [27]: # Gradient Boosting
model_gb = GradientBoostingClassifier(n_estimators=100, random_state=42)
model_gb, metrics_gb, y_tr_gb, y_te_gb = evaluate_sklern_model(
    model_gb, 'Gradient Boosting', XTknn, Ytrain, XVknn, Ytest, cv=cv)
```





|   | Model             | Set   | Cut-off | Accuracy | Precision | Sensitivity | Specificity | F1-score | AUC      | MW-U p-value |
|---|-------------------|-------|---------|----------|-----------|-------------|-------------|----------|----------|--------------|
| 0 | Gradient Boosting | Train | 0.5     | 1.000000 | 1.000000  | 1.000000    | 1.0         | 1.000000 | 1.000000 | 1.003666e-11 |
| 1 | Gradient Boosting | Test  | 0.5     | 0.857143 | 0.833333  | 0.909091    | 0.8         | 0.869565 | 0.918182 | 1.273677e-03 |

### Gradient Boosting 5-fold CV ROC

AUC = 0.791. Rendimiento moderado.

**Gradient Boosting Train/Test** Train: AUC = 1.000. Test: AUC = 0.918.

**Tabla:** Test Accuracy 0.857, Sensibilidad 0.909, Especificidad 0.80, F1 0.870, AUC 0.918.

**Interpretación:** Buen modelo, aunque el CV no fue tan alto. En test funciona bien.

## Comparación Global y Exportación

### Tabla Comparativa de Métricas

```
In [28]: all_metrics = pd.concat([
    metrics_pls, metrics_lin, metrics_log, metrics_tree,
    metrics_svm, metrics_knn, metrics_nb, metrics_rf, metrics_gb
], ignore_index=True)
display(all_metrics)
```

|    | Model               | Set   | Cut-off | Accuracy | Precision | Sensitivity | Specificity | F1-score | AUC      | MW-U p-value |
|----|---------------------|-------|---------|----------|-----------|-------------|-------------|----------|----------|--------------|
| 0  | PLS-DA              | Train | 0.5     | 0.887097 | 0.903226  | 0.875000    | 0.9         | 0.888889 | 0.967708 | 2.657802e-10 |
| 1  | PLS-DA              | Test  | 0.5     | 0.761905 | 0.800000  | 0.727273    | 0.8         | 0.761905 | 0.945455 | 6.371949e-04 |
| 2  | Linear Regression   | Train | 0.5     | 1.000000 | 1.000000  | 1.000000    | 1.0         | 1.000000 | 1.000000 | 1.436343e-11 |
| 3  | Linear Regression   | Test  | 0.5     | 0.714286 | 0.777778  | 0.636364    | 0.8         | 0.700000 | 0.772727 | 3.777135e-02 |
| 4  | Logistic Regression | Train | 0.5     | 1.000000 | 1.000000  | 1.000000    | 1.0         | 1.000000 | 1.000000 | 1.436343e-11 |
| 5  | Logistic Regression | Test  | 0.5     | 0.809524 | 0.818182  | 0.818182    | 0.8         | 0.818182 | 0.881818 | 3.474099e-03 |
| 6  | Decision Tree       | Train | 0.5     | 1.000000 | 1.000000  | 1.000000    | 1.0         | 1.000000 | 1.000000 | 6.087861e-15 |
| 7  | Decision Tree       | Test  | 0.5     | 0.952381 | 0.916667  | 1.000000    | 0.9         | 0.956522 | 0.950000 | 5.795438e-05 |
| 8  | SVM                 | Train | 0.5     | 1.000000 | 1.000000  | 1.000000    | 1.0         | 1.000000 | 1.000000 | 1.436343e-11 |
| 9  | SVM                 | Test  | 0.5     | 0.809524 | 0.818182  | 0.818182    | 0.8         | 0.818182 | 0.836364 | 1.016244e-02 |
| 10 | KNN                 | Train | 0.5     | 0.854839 | 0.828571  | 0.906250    | 0.8         | 0.865672 | 0.916146 | 7.626721e-09 |
| 11 | KNN                 | Test  | 0.5     | 0.857143 | 0.900000  | 0.818182    | 0.9         | 0.857143 | 0.859091 | 5.147835e-03 |
| 12 | Naive Bayes         | Train | 0.5     | 0.854839 | 0.896552  | 0.812500    | 0.9         | 0.852459 | 0.923958 | 1.024385e-08 |
| 13 | Naive Bayes         | Test  | 0.5     | 0.714286 | 0.777778  | 0.636364    | 0.8         | 0.700000 | 0.836364 | 1.016244e-02 |
| 14 | Random Forest       | Train | 0.5     | 1.000000 | 1.000000  | 1.000000    | 1.0         | 1.000000 | 1.000000 | 1.432978e-11 |
| 15 | Random Forest       | Test  | 0.5     | 0.904762 | 0.909091  | 0.909091    | 0.9         | 0.909091 | 0.963636 | 3.763967e-04 |
| 16 | Gradient Boosting   | Train | 0.5     | 1.000000 | 1.000000  | 1.000000    | 1.0         | 1.000000 | 1.000000 | 1.003666e-11 |
| 17 | Gradient Boosting   | Test  | 0.5     | 0.857143 | 0.833333  | 0.909091    | 0.8         | 0.869565 | 0.918182 | 1.273677e-03 |

**Tabla Comparativa de Métricas** Resume todos los modelos en Train y Test.

**Mejores en Test (AUC):****Random Forest: 0.964**

Decision Tree: 0.950

**PLS-DA: 0.945**

Gradient Boosting: 0.918

Logistic Regression: 0.882

KNN: 0.859

SVM: 0.836

Naive Bayes: 0.836

Linear Regression: 0.773

**Pero ojo:** Decision Tree tiene CV ROC de solo 0.645, así que su buen test puede ser engañoso. Random Forest y PLS-DA son más consistentes.

## Curvas ROC Superpuestas (Test Set)

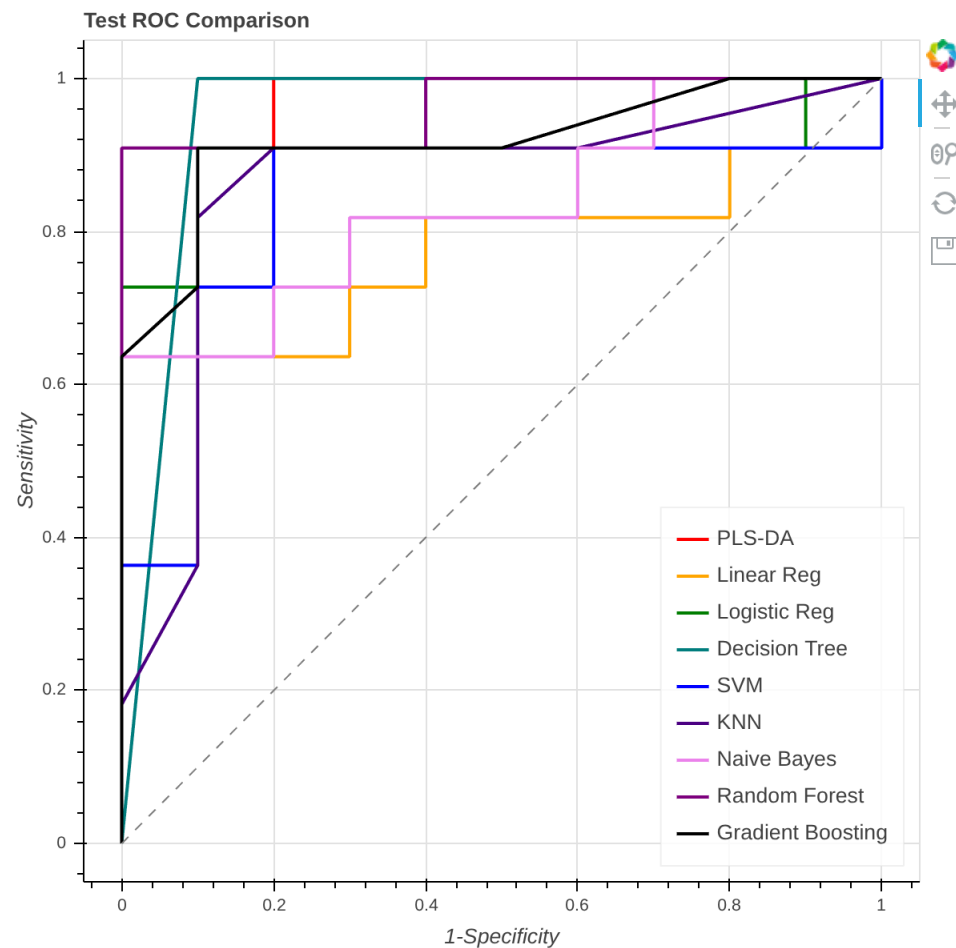
```
In [29]: models_dict = {
    'PLS-DA': YVpred_pls, 'Linear Reg': y_te_lin, 'Logistic Reg': y_te_log,
    'Decision Tree': y_te_tree, 'SVM': y_te_svm, 'KNN': y_te_knn,
    'Naive Bayes': y_te_nb, 'Random Forest': y_te_rf, 'Gradient Boosting': y_te_gb
}

colors = ['red', 'orange', 'green', 'teal', 'blue', 'indigo', 'violet', 'purple', 'black']

p = figure(title='Test ROC Comparison', plot_width=600, plot_height=600,
           tools='pan,wheel_zoom,reset,save')
p.xaxis.axis_label = '1-Specificity'
p.yaxis.axis_label = 'Sensitivity'

for (name, y_sc), color in zip(models_dict.items(), colors):
    y_sc = np.asarray(y_sc, dtype=float)
    mask = np.isfinite(y_sc)
    fpr, tpr, _ = roc_curve(Ytest[mask], y_sc[mask])
    src = ColumnDataSource(data=dict(fpr=fpr, tpr=tpr))
    p.line('fpr', 'tpr', source=src, line_color=color, line_width=2, legend=name)

p.line([0, 1], [0, 1], line_color='gray', line_dash='dashed')
p.legend.location = 'bottom_right'
show(p)
```



## Exportar a Excel

```
In [30]: prediction_sheet = dataTest[["Idx", "SampleID", "Class"]].reset_index(drop=True).copy()
prediction_sheet['PLS_score'] = YVpred_pls
prediction_sheet['LinReg_score'] = y_te_lin
prediction_sheet['LogReg_score'] = y_te_log
prediction_sheet['Tree_score'] = y_te_tree
prediction_sheet['SVM_score'] = y_te_svm
prediction_sheet['KNN_score'] = y_te_knn
prediction_sheet['NB_score'] = y_te_nb
prediction_sheet['RF_score'] = y_te_rf
prediction_sheet['GB_score'] = y_te_gb

for col, scores in [('PLS_pred', YVpred_pls), ('LinReg_pred', y_te_lin),
```

```

        ('LogReg_pred', y_te_log), ('Tree_pred', y_te_tree),
        ('SVM_pred', y_te_svm), ('KNN_pred', y_te_knn),
        ('NB_pred', y_te_nb), ('RF_pred', y_te_rf),
        ('GB_pred', y_te_gb)]:
    prediction_sheet[col] = np.where(scores >= 0.5, 'GC', 'HE')

display(prediction_sheet)

writer = pd.ExcelWriter("Metabolomics_ML_Comparison.xlsx")
prediction_sheet.to_excel(writer, sheet_name='Predictions', index=False)
all_metrics.to_excel(writer, sheet_name='Metrics_Summary', index=False)
writer.save()
print("Done!")

```

|    | Idx | SampleID   | Class | PLS_score | LinReg_score | LogReg_score | Tree_score | SVM_score | KNN_score | NB_score     | ... | GB_score | PLS_pred | LinReg_pred |
|----|-----|------------|-------|-----------|--------------|--------------|------------|-----------|-----------|--------------|-----|----------|----------|-------------|
| 0  | 4   | sample_4   | HE    | 0.596320  | 0.919904     | 0.851315     | 0.0        | 0.847171  | 0.8       | 4.428249e-01 | ... | 0.000262 | GC       | GC          |
| 1  | 78  | sample_78  | GC    | 0.762340  | 0.149727     | 0.894750     | 1.0        | 0.679352  | 0.6       | 4.679933e-01 | ... | 0.999781 | GC       | HE          |
| 2  | 90  | sample_90  | HE    | 0.190719  | -0.324845    | 0.098203     | 1.0        | 0.234711  | 0.2       | 5.714335e-01 | ... | 0.999585 | HE       | HE          |
| 3  | 71  | sample_71  | GC    | 0.799896  | 1.137838     | 0.925095     | 1.0        | 0.793073  | 1.0       | 2.097966e-02 | ... | 0.999464 | GC       | GC          |
| 4  | 92  | sample_92  | GC    | 1.046995  | 1.307476     | 0.989177     | 1.0        | 0.950153  | 0.6       | 9.988925e-01 | ... | 0.999781 | GC       | GC          |
| 5  | 119 | sample_119 | HE    | 0.122116  | -0.088174    | 0.004887     | 0.0        | 0.026319  | 0.0       | 4.147888e-03 | ... | 0.000240 | HE       | HE          |
| 6  | 56  | sample_56  | HE    | 0.166406  | -0.004713    | 0.039297     | 0.0        | 0.261738  | 0.2       | 5.392801e-08 | ... | 0.000262 | HE       | HE          |
| 7  | 104 | sample_104 | HE    | -0.130498 | -0.658361    | 0.001523     | 0.0        | 0.038638  | 0.2       | 6.108907e-13 | ... | 0.000262 | HE       | HE          |
| 8  | 98  | sample_98  | GC    | 0.230983  | -0.476890    | 0.144049     | 1.0        | 0.426947  | 0.6       | 1.000000e+00 | ... | 0.000262 | HE       | HE          |
| 9  | 36  | sample_36  | GC    | 0.401337  | 1.365829     | 0.698362     | 1.0        | 0.906146  | 0.4       | 2.376365e-06 | ... | 0.999760 | HE       | GC          |
| 10 | 35  | sample_35  | HE    | 0.050801  | 0.143800     | 0.009956     | 0.0        | 0.054162  | 0.0       | 6.506793e-02 | ... | 0.000240 | HE       | HE          |
| 11 | 95  | sample_95  | GC    | 1.135492  | 1.524514     | 0.990347     | 1.0        | 0.844571  | 1.0       | 9.999999e-01 | ... | 0.999782 | GC       | GC          |
| 12 | 94  | sample_94  | HE    | -0.136688 | 0.179527     | 0.003185     | 0.0        | 0.071594  | 0.2       | 3.515781e-07 | ... | 0.000266 | HE       | HE          |
| 13 | 129 | sample_129 | HE    | 0.727619  | 0.849387     | 0.847724     | 0.0        | 0.602117  | 0.4       | 9.525064e-01 | ... | 0.001967 | GC       | GC          |
| 14 | 58  | sample_58  | GC    | 1.069837  | 1.163263     | 0.978438     | 1.0        | 0.768251  | 0.8       | 9.992812e-01 | ... | 0.999689 | GC       | GC          |
| 15 | 75  | sample_75  | GC    | 1.165530  | 2.327686     | 0.997801     | 1.0        | 0.965260  | 0.8       | 9.999993e-01 | ... | 0.999782 | GC       | GC          |
| 16 | 63  | sample_63  | HE    | 0.180942  | 0.426997     | 0.046769     | 0.0        | 0.149097  | 0.0       | 5.547905e-02 | ... | 0.997045 | HE       | HE          |
| 17 | 88  | sample_88  | GC    | 1.014826  | 1.085767     | 0.967370     | 1.0        | 0.561439  | 0.6       | 9.981578e-01 | ... | 0.999133 | GC       | GC          |
| 18 | 101 | sample_101 | HE    | 0.174986  | -1.817412    | 0.012445     | 0.0        | 0.034711  | 0.0       | 3.155717e-02 | ... | 0.419114 | HE       | HE          |
| 19 | 43  | sample_43  | GC    | 0.392416  | -3.012745    | 0.001900     | 1.0        | 0.004753  | 0.0       | 1.554890e-01 | ... | 0.999585 | HE       | HE          |
| 20 | 33  | sample_33  | GC    | 0.979422  | 0.291893     | 0.988910     | 1.0        | 0.860331  | 0.6       | 9.951447e-01 | ... | 0.999782 | GC       | HE          |

21 rows × 21 columns

```
In [32]: !jupyter nbconvert --to pdf Metabolomics3.ipynb
```



```
[NbConvertApp] Converting notebook Metabolomics3.ipynb to pdf
/home/alex/miniconda3/envs/metabolomics/lib/python3.7/site-packages/nbconvert/utils/pandoc.py:51: RuntimeWarning: You are using an unsupported version of pandoc (2.9.2.1).
Your version must be at least (2.14.2) but less than (4.0.0).
Refer to https://pandoc.org/installing.html.
Continuing with doubts...
  check_pandoc_version()
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime type(s) dict_keys(['text/html']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime type(s) dict_keys(['application/javascript', 'application/vnd.bokehjs_load.v0+json']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime type(s) dict_keys(['text/html']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime type(s) dict_keys(['application/javascript', 'application/vnd.bokehjs_load.v0+json']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime type(s) dict_keys(['application/javascript', 'application/vnd.bokehjs_exec.v0+json']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime type(s) dict_keys(['text/html']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime type(s) dict_keys(['application/javascript', 'application/vnd.bokehjs_load.v0+json']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime type(s) dict_keys(['application/javascript', 'application/vnd.bokehjs_exec.v0+json']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime type(s) dict_keys(['text/html']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime type(s) dict_keys(['application/javascript', 'application/vnd.bokehjs_exec.v0+json']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime type(s) dict_keys(['text/html']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime type(s) dict_keys(['application/javascript', 'application/vnd.bokehjs_exec.v0+json']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime type(s) dict_keys(['text/html']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime type(s) dict_keys(['application/javascript', 'application/vnd.bokehjs_exec.v0+json']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime type(s) dict_keys(['text/html']) is not able to be represented.
  ((*- block data_text -*))
```

```

/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime
type(s) dict_keys(['application/javascript', 'application/vnd.bokehjs_exec.v0+json']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime
type(s) dict_keys(['text/html']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime
type(s) dict_keys(['application/javascript', 'application/vnd.bokehjs_exec.v0+json']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime
type(s) dict_keys(['text/html']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime
type(s) dict_keys(['application/javascript', 'application/vnd.bokehjs_exec.v0+json']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime
type(s) dict_keys(['text/html']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime
type(s) dict_keys(['application/javascript', 'application/vnd.bokehjs_exec.v0+json']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime
type(s) dict_keys(['text/html']) is not able to be represented.
  ((*- block data_text -*))
/home/alex/miniconda3/envs/metabolomics/share/jupyter/nbconvert/templates/latex/display_priority.j2:31: UserWarning: Your element with mime
type(s) dict_keys(['application/javascript', 'application/vnd.bokehjs_exec.v0+json']) is not able to be represented.
  ((*- block data_text -*))
[NbConvertApp] Writing 111878 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 89892 bytes to Metabolomics3.pdf

```

```
In [ ]: !jupyter nbconvert --to html Metabolomics3.ipynb
```